

Bocconi

COMPUTER SCIENCE

code 30424 a.y. 2024-2025

Lesson 21

Deepening some relevant libraries: the NLTK Module



Universit 
Bocconi
MILANO

Objectives of the lesson

- Learn how use the Natural Language Toolkit (NLTK) module

NLTK module

NLTK (Natural Language Toolkit) is a **Python non-standard library** that provides useful tools to read, understand, and work with **human language**, like English, Italian, etc.

It's often used in **NLP (Natural Language Processing)**, the branch of **artificial intelligence** that focuses on the interaction between computers and humans, empowering computers to understand, interpret, and generate human language in a meaningful way.

With the tools provided by **NLTK** it comes easy in Python to:

- Split text into words and sentences
- Clean up text (remove punctuation, common words, etc.)
- Find the meaning of words (like in a dictionary)
- Identify grammar parts (nouns, verbs, etc.)
- Analyze and classify text (e.g. positive or negative sentiment)

NLTK module

Before start using **NLTK** for the first time, it is essential to download the module files and some **language data** on our computer (the following operations have to be done only once):

- install **NLTK** using the **pip** installation procedure from the CMD (or Terminal) window
- install the most common **language data** executing these commands from the **IDLE shell**:

```
>>> import nltk
>>> nltk.download('punkt')           # for tokenization
>>> nltk.download('punkt_tab')       # for tokenization
>>> nltk.download('stopwords')       # for stopword removal
>>> nltk.download('wordnet')         # for WordNet
>>> nltk.download('omw-1.4')         # for Open Multilingual Wordnet
>>> nltk.download('vader_lexicon')   # for Sentiment Analysis
```

Tokenization

Tokenization in **NLTK** refers to the process of breaking text into smaller units called **tokens**, such as words, sub-words, or characters, to make it easier for a computer to analyze and process language data.

Here are some useful commands and methods:

Command or method	Description
<code>MySList = nltk.tokenize.sent_tokenize('text')</code>	Returns a sentence-tokenized copy of 'text' as a list of string (defining the instance <i>MySList</i> as an object of class list)
<code>MyWList = nltk.tokenize.word_tokenize('text')</code>	Returns a word-tokenized copy of 'text' as a list of string (defining the instance <i>MyWList</i> as an object of class list)

Note: for both commands above, the “**tokenizer**” used to split the sentence 'text' depends on the language defined by the user (by default “english”) and it includes punctuations symbols other than just the space (as word tokenizer) and the dot “.” (as sentence tokenizer)

Tokenization example

Given the variable **MySentence** containing the text:

In our society, owning programming skills is increasingly important. This applies not only to those working in the IT sector, but also to those working in fields such as science, art, education, and business management. Learning to program is indeed important not only because it allows to develop applications to support business activities, but above all because it allows to focus on the most effective approach to problem solving, combining technical aspects with logic and creativity.

This text aims to enable even those who have never approached programming before to write code in Python, starting from the fundamental elements of syntax and progressing to the understanding how to expand Python to adapt to specific studying or working needs. The text addresses key concepts such as data types, variables, conditional expressions, loops, custom functions, exception handling, classes and objects, the Standard Library modules and third-party modules.

The following script returns this output: 

```
import nltk

MySentence = "In our society, owning programming skills:

MySList = nltk.tokenize.sent_tokenize(MySentence)
MyWList = nltk.tokenize.word_tokenize(MySentence)

for MyS in MySList:
    print(MyS + '\n')

print(MyWList)
```

In our society, owning programming skills is increasingly important.

This applies not only to those working in the IT sector, but also to those working in fields such as science, art, education, and business management.

Learning to program is indeed important not only because it allows to develop applications to support business activities, but above all because it allows to focus on the most effective approach to problem solving, combining technical aspects with logic and creativity.

This text aims to enable even those who have never approached programming before to write code in Python, starting from the fundamental elements of syntax and progressing to the understanding how to expand Python to adapt to specific studying or working needs.

The text addresses key concepts such as data types, variables, conditional expressions, loops, custom functions, exception handling, classes and objects, the Standard Library modules and third-party modules.

```
['In', 'our', 'society', ',', 'owning', 'programming', 's', 'kills', 'is', 'increasingly', 'important', '.', 'This', 'applies', 'not', 'only', 'to', 'those', 'working', 'in', 'the', 'IT', 'sector', ',', 'but', 'also', 'to', 'those', 'working', 'in', 'fields', 'such', 'as', 'science', ',', 'art', ',', 'education', ',', 'and', 'business', 'management', '.', 'Learning', 'to', 'program', 'is', 'indeed', 'important', 'not', 'only', 'because', 'it', 'allows', 'to', 'develop', 'applications', 'to', 'support', 'business', 'activities', ',', 'but', 'above', 'all', 'because', 'it', 'allows', 'to', 'focus', 'on', 'the', 'most', 'effective', 'approach', 'to', 'problem', 'solving', ',', 'combining', 'technical', 'aspects', 'with', 'logic', 'and', 'creativity', '.', 'This', 'text', 'aims', 'to', 'enable', 'even', 'those', 'who', 'have', 'never', 'approached', 'programming', 'before', 'to', 'write', 'code', 'in', 'Python', ',', 'starting', 'from', 'the', 'fundamental', 'elements', 'of', 'syntax', 'and', 'progressing', 'to', 'the', 'understanding', 'how', 'to', 'expand', 'Python', 'to', 'adapt', 'to', 'specific', 'studying', 'or', 'working', 'needs', '.']
```



Stopwords

Thanks to the **language data** component '**stopwords**' (that can be downloaded on the computer of the user with the command "nltk.download"), with **NLTK** is possible to remove from a lists of words all common words (e.g. "is", "the", "and", ...) which usually don't add much meaning.

Here is one relevant command:

Command or method	Description
<code>MyStopW = nltk.corpus.stopwords.words('language')</code>	Returns a list with all the stopwords of the language provided as argument (if 'language' argument is not defined, the list of all known stopwords is returned)

Note: the following approach in the syntax is commonly used to improve on readability:

```
from nltk.corpus import stopwords

MyENstop = stopwords.words('english')

MyITstop = stopwords.words('italian')
```

In **NLTK**, a **corpus** is a collection of text documents or datasets used for natural language processing tasks, often organized in directories and accessible via specialized corpus reader functions for analysis

Stopwords exercise

The file **Stopwords.py** contains the variable `MyThought` and the statements to import the modules needed to the program.

Following the suggested approach, you have to complete the script so to print on screen the list of the most significant words present in the string **MyThought**.

Considering:

```
MyThought = 'We are all here to learn Python so to pass Computer science exam'
```

The resulting list must be:

```
['learn', 'python', 'pass', 'computer', 'science', 'exam']
```


Frequency distribution

To define the **frequency distribution** of each word in a sentence we need to count how many times each word appears in the sentence. The **FreqDist** class available in **NLTK** is used to count the frequency of elements (usually words or tokens) in a dataset.

Here are some useful commands and methods:

Command or method	Description
<code>MyFreq = nltk.FreqDist(dataset)</code>	Returns a frequency distribution object, which is essentially a specialized Python dictionary where Keys are the elements of the dataset (e.g. characters / words / tokens) and Values are the frequency counts of those elements. The argument <i>dataset</i> can be a sequence.
<code>MyFreq.max()</code>	Returns the element with the greatest number of occurrences in the frequency distribution represented by <i>MyFreq</i> . If two or more elements have the same number of occurrences, it returns one of them (which element is returned is undefined).
<code>MyFreq.most_common(n=None)</code>	Returns a list of tuples with the <i>n</i> most common elements and their counts from the most common to the least. If <i>n</i> is None it returns all element counts.

Frequency distribution exercise

The file **Frequency.py** contains a script with some functions and some variables already available. Following the approach suggested, you have to:

- Complete **CharFreq02** function using **NLTK** functionalities
- Complete **WordFreq01** function using methods of **dictionaries**
- Integrate the functions calls adding those just completed
- Call the **MostFreq** function in the IDLE Shell, providing as argument a different sentence than those available in the sample variables

Dictionary and Thesaurus

The **WordNet** language dataset in **NLTK** is a large lexical database that organizes English words into sets of synonyms called **synsets**.

Each **synset** represents a specific meaning of a word and includes related information such as definitions, examples, synonyms, related words, and more.

WordNet allows to explore conceptual relationships between words and perform tasks like finding synonyms, analyzing word senses, and identifying semantic similarities.

WordNet supports multiple languages through the **Open Multilingual Wordnet (OMW)** project, enabling cross-linguistic analysis

Some of the benefits that it offers:

- ✓ Helps in making chatbots smarter.
- ✓ Can improve search results by understanding word meanings.
- ✓ Useful for basic text analysis, like finding how words are connected.

Dictionary and Thesaurus

WordNet offers various objects and methods that rely on these definitions:

- **Synset**
short for “Synonym Set” is a group of words that all **mean the same thing** in a certain context (i.e. a synset represent “one meaning”)
- **Lemmas**
each synset has **lemmas**, which are the **individual words** that are synonyms for that meaning (i.e. a lemma is a word that expresses the meaning represented by the related synset)
- **Definition**
each synset comes with a **definition**, that is a plain-English explanation of what it means

Dictionary and Thesaurus

Here are some useful commands and methods:

Command or method	Description
<code>MySyn = nltk.corpus.wordnet.synsets('word')</code>	Returns a list of Synset objects, each of which is like a «package» that contains everything about a word (definition, synonyms, examples, etc.)
<code>MySyn[n].name()</code>	Returns a string representing the ID of the element with index n of the synset, including: the word, the part of speech (noun, verb, adjective, ...) and the index
<code>MySyn[n].definition()</code>	Returns a string representing the meaning of the element with index n of the synset
<code>MySyn[n].examples()</code>	Returns a list of strings with real-world example sentences of the element with index n of the synset
<code>MySyn[n].lemmas()</code>	Returns a list of Lemma objects, each one containing a word that means the same thing (synonym) of the element with index n of the synset
<code>MySyn[n].lemmas()[m].name()</code>	Returns a string representing the name (i.e. the word) of the lemma with index m belonging to the element with index n of the synset



Dictionary and Thesaurus exercise

The file **Wordnet.py** contains a script with a function to complete. Following the approach suggested, you have to:

- Complete the **explain** function so that all **definitions** of the argument “MyWord” are printed on screen as a numbered list (see the result with the word ‘happy’ here below)

```
>>> explain('happy')  
1. enjoying or showing or marked by joy or pleasure  
2. marked by good fortune  
3. eagerly disposed to act or to be of service  
4. well expressed and to the point
```

Sentiment analysis

Thanks to the **NLTK** module **VADER** (Valence Aware Dictionary and sEntiment Reasoner) – available as part of the **nltk.sentiment** package – it is possible to use the tools of the **SentimentIntensityAnalyzer** class which is specifically designed for sentiment analysis.

Sentiment analysis is the process of determining whether a sentence expresses a positive, negative, or neutral sentiment.

Sentiment analysis using NLTK is widely used for tasks like:

- customer feedback analysis
- social media monitoring
- brand reputation management

Sentiment analysis

When processing a sentence, the SentimentIntensityAnalyzer:

- uses a predefined sentiment lexicon in which each word is assigned a sentiment score based on polarity (positive, negative or neutral) and intensity. Words valence score ranges from -4 (most extreme negative sentiment) to +4 (most extreme positive sentiment)
- takes into account the context using rules for punctuation (e.g., exclamation marks), capitalization (e.g., "GREAT"), degree modifiers (e.g., "very"), and negations (e.g., "not good"), using also emojis and slang to adjust the sentiment score
- calculates the sentence compound score, which is a normalized, weighted sum of the valence scores of each word. This score ranges from -1 (most negative) to +1 (most positive)

Sentiment analysis

To access the SentimentIntensityAnalyzer tools, the corresponding class of the VADER module must be previously imported using the syntax:

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer
```

Here are some useful commands and methods:

Command or method	Description
<code>MySent = SentimentIntensityAnalyzer()</code>	Returns a SentimentIntensityAnalyzer object containing the sentiment lexicon and various methods to analyze the text
<code>MySent.polarity_scores('text')</code>	Returns a dictionary object with sentiment scores for positive (pos), negative (neg), neutral (neu), and compound (compound) sentiments of the 'text' argument

Sentiment analysis example

Providing to the `sent(MyText)` function the text:

`'I like Python'`

The following script returns this output:



0.0	neg
0.286	neu
0.714	pos
0.3612	compound

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer
```

```
def sent(MyText):  
    MySent = SentimentIntensityAnalyzer()  
    MyScores = MySent.polarity_scores(MyText)
```

```
    for k, v in MyScores.items():  
        print(v, k, sep = "\t")
```



Tools to learn with today's lecture

- Understand the basics of Natural Language Processing
- Learn how to use some entry functionalities of NLTK module

Book references

Understanding Python:

Chapters 13.8